



UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

Corso di Laurea in Sicurezza dei Sistemi e delle Reti Informatiche

Recupero e implementazione di
componenti ML per analisi dati in un
contesto distribuito

Relatore

Prof. Claudio A. Ardagna

Correlatori

Prof. Marco Anisetti

Dott. Antongiaco Polimeno

Tesi di laurea di

Gabriele Stentella

984491

Anno Accademico 2023/2024

Prefazione

Fra il 2003 e il 2006, Google divulgò tre componenti fondamentali sviluppati per affrontare la sfida di indicizzare internet: la tecnologia del proprio file system, il modello MapReduce e il sistema di archiviazione dati distribuito Bigtable; rivoluzionando l'industria nascente dei Big Data e gettando le basi per lo sviluppo del framework *Hadoop*, uno strumento *open source* che ha consentito una larga adozione di tecniche per la computazione distribuita. Negli anni successivi, molte di quelle che ora sono le grandi aziende tecnologiche statunitensi hanno adottato e sviluppato l'ecosistema circostante *Hadoop*, implementando diversi file system distribuiti e framework per convertire le classiche interrogazioni *SQL* in lavori MapReduce. Dal momento che avviare e gestire un cluster *Hadoop* era un'operazione costosa e impegnativa, la vera adozione di massa dell'infrastruttura è avvenuta quando Amazon lanciò il suo servizio di cloud computing *on-demand* e contestualmente *Elastic MapReduce (EMR)*, consentendo di sfruttare i vantaggi del modello MapReduce impiegando il file system distribuito *Amazon S3* senza dover acquistare e configurare l'hardware necessario.

Nel 2010 Google rilasciò una nuova ricerca in cui veniva illustrata un'architettura per *query* distribuite e un formato di archiviazione utile a rendere efficienti le operazioni, ancora una volta ispirando altre aziende a sviluppare e poi rendere *open source* tecnologie come *Apache Impala*, *Presto* e *Apache Parquet*.

Alcuni anni dopo vennero rilasciate le prime versioni di *Apache Spark* che ottennero da subito un grande successo, infatti era molto semplice migrare da sistemi per interrogazioni *SQL* di *Hadoop* a *SparkSQL*, e il nuovo framework aveva una sintassi più semplice e migliori performance di MapReduce, grazie alla sua capacità di utilizzare la RAM come cache. La gestione delle risorse del cluster era gestita attraverso il *resource manager Hadoop Yarn*. Poco dopo *Spark* cominciò ad avere successo anche nel mondo del Machine Learning (ML), grazie al framework *Apache Mahout* prima, e poi alla *Machine Learning library (MLlib)*, più veloce; inoltre le possibilità offerte da *Spark* si ampliarono grazie alla nascita di scheduler come *Apache Airflow* e *Luigi*.

Fra il 2013 e il 2014 si diffusero *Docker* e l'orchestratore *Kubernetes* che sem-

plificarono la gestione di cluster anche di grandi dimensioni, rendendo le architetture molto spiù stabili e scalabili; *Spark* si è adattato al contesto, permettendo di utilizzare proprio *Kubernetes* come *resource manager*.

Con l'avvento del Deep Learning (DL) le attività di computazione si sono spostate sull'utilizzo massiccio di GPU e sull'impiego delle più importanti librerie di *Python* dedicate, *Tensorflow* e *Pytorch*, dunque per questi carichi di lavoro si è passati all'utilizzo delle Application Programming Interface (API) proprietarie delle librerie per distribuire la computazione anziché usare *Spark*, in modo da usare in maniera efficiente la GPU ed evitare di dover creare un cluster per poi installare le librerie su tutti i componenti.

Il percorso che si intende illustrare in questo elaborato ha previsto sia una fase di recupero di algoritmi ML eseguiti in un ambiente distribuito attraverso *Spark*, sia l'implementazione di nuovi elementi che sfruttano le API delle moderne librerie DL, nell'introduzione viene analizzato lo scenario di partenza che rappresenta il contesto in cui si inserisce il lavoro di tesi, in modo tale da illustrare nel dettaglio le motivazioni alla base dell'attività svolta, per poi descrivere il contributo dato dalla soluzione proposta e dettagliare la struttura della trattazione che segue.

Indice

1	Le tecnologie per l'analisi dati e il calcolo distribuito	3
2	Gli algoritmi di analisi dati	4
3	Recupero e implementazione degli algoritmi	10
4	Conclusioni	13
A	POM generale	14
B	POM di un singolo modulo	15
	Bibliografia	23

Elenco delle figure

Introduzione

La presenza sempre più diffusa dei *Big Data* ha reso necessario avere infrastrutture e tecnologie di analisi adatte ad estrarre valore da tali dati e che siano scalabili per essere utilizzate da realtà di diverse dimensioni. Al fine di avere procedure efficaci sia nell'attività di analisi che in quella di costruzione di modelli predittivi, occorre affrontare i problemi tipici dei *Big Data* che sono la diffusa eterogeneità, l'accumulo di rumore in fase di raccolta, la frequente presenza di correlazioni spurie. Gli algoritmi impiegati devono essere anche accurati dal punto di vista statistico ed efficaci per quanto riguarda il carico computazionale; per tutte queste ragioni, ma in modo particolare per la necessità di efficienza computazionale, i *Big Data* hanno portato allo sviluppo di nuove infrastrutture di computazione e nuovi metodi di memorizzazione dei dati [8].

In tale contesto, le tecnologie di *Apache Hadoop* prima e *Apache Spark* poi, hanno reso semplice ed economica l'analisi e la memorizzazione dei dataset di grandi dimensioni, in particolare, dal momento che l'analisi di *Big Data* richiede una sperimentazione reiterata per trovare i parametri corretti e individuare le giuste metriche di valutazione, *Spark* rappresenta una soluzione tecnologicamente adatta a supportare in modo efficiente questo tipo di lavoro, in particolare se viene utilizzata la libreria MLlib per il *design* e il *tuning* di algoritmi ML [14].

Negli ultimi anni, oltre agli algoritmi ML di *shallow learning*, hanno avuto molto successo i modelli DL che, a differenza delle architetture *shallow*, traggono vantaggio dall'avere un grande numero di parametri liberi, quindi l'addestramento di tali modelli vede nei *Big Data* una risorsa fondamentale, e contemporaneamente necessita di metodi efficienti per la parallelizzazione delle operazioni di *training* [23].

I benefici dati da un'analisi dati svolta con l'ausilio di algoritmi ML o modelli DL sono evidenti nel campo della ricerca medica e scientifica, nella gestione di infrastrutture critiche e nella cosiddetta *business intelligence* [11], ma la letteratura è sempre più ricca di applicazioni di queste tecnologie nel campo della sicurezza informatica: grazie alla presenza pervasiva dell'Internet of Things

(IoT) e in generale dell'evoluzione tecnologica dei dispositivi informatici, compresi gli strumenti di rete, c'è una grandissima produzione di dati che possono essere analizzati in tempo reale per eseguire azioni di prevenzione, ricerca di minacce e *incident response*. Soltanto alcune fra le numerose applicazioni in questo campo sono l'analisi del traffico di rete con l'obiettivo di istituire uno schema di priorità e rendere meno efficaci eventuali attacchi Denial of Service (DoS), identificare comportamenti sospetti, individuare malware e integrare i sistemi di autenticazione [15]. In particolare, l'uso dei classificatori è utile per implementare dei filtri sia sul traffico web sia sulla corrispondenza di posta elettronica, in modo tale da ridurre la probabilità che il personale di un'azienda entri in contatto con contenuti malevoli, e di conseguenza minimizzare l'impatto di email spam o siti web malevoli sulla rete aziendale o di un'infrastruttura critica; i classificatori in questione devono essere allenati con un insieme di dati molto grande e in continuo aggiornamento, in modo tale che stiano al passo con l'evoluzione delle minacce, inoltre è necessario che, in base al tipo di classificazione, venga utilizzato un modello statistico adeguato.

L'obiettivo del lavoro svolto durante il tirocinio interno, è quello di recuperare degli algoritmi ML di analisi dati che sono stati sviluppati in linguaggio Java e utilizzano la tecnologia *Spark*, con il fine di renderli eseguibili in un ambiente distribuito, superando il *resource manager* originario Hadoop Yarn e utilizzando Kubernetes. Parallelamente al recupero degli algoritmi esistenti, la tesi si pone l'obiettivo di studiare la tecnologia di esecuzione distribuita della libreria *Tensorflow* per lo sviluppo di nuovi algoritmi per l'analisi dati eseguibili nel medesimo ambiente distribuito dei precedenti.

L'elaborato è organizzato come segue: nel primo capitolo si espone lo stato dell'arte delle tecnologie per l'analisi dati e il calcolo distribuito, in modo tale da introdurre gli aspetti tecnici alla base del lavoro svolto; nel secondo capitolo si illustrano gli algoritmi coinvolti nella tesi; nel terzo capitolo viene descritto nel dettaglio il lavoro svolto, seguendo la divisione del lavoro nelle fasi di *development*, *deployment* e *testing*; nel quarto e ultimo capitolo si riassume il contributo dato da questa tesi e si esplorano i possibili sviluppi futuri.

Capitolo 1

Le tecnologie per l'analisi dati e il calcolo distribuito

Capitolo 2

Gli algoritmi di analisi dati

Una parte fondamentale del lavoro svolto è stata quella di recuperare degli algoritmi ML e statistici sviluppati in linguaggio Java, sfruttando le librerie offerte da Spark, di seguito si descrivono in dettaglio le tecniche di inferenza e gli algoritmi implementati dai task.

K-means clustering K-means è un algoritmo di clustering con approccio partizionale, ottiene in input il parametro k , numero di cluster, poi la computazione inizia selezionando k centroidi casuali dal dataset su cui è eseguito, viene calcolata la distanza fra ogni punto del dataset e i centroidi, in modo da assegnare ogni dato al cluster relativo al centroide più vicino. Ogni volta che un nuovo punto viene assegnato al cluster, il centro viene ricalcolato, quindi l'algoritmo viene eseguito iterativamente finché i cluster non sono stabili (nessun punto viene più assegnato a un nuovo cluster) [10]. Osservando il clustering come un *learning problem* — ovvero trattandolo come un compito di apprendimento automatico in cui l'obiettivo è assegnare automaticamente gli oggetti in gruppi omogenei — si utilizza l'algoritmo K-means per allenare un modello ML che possa essere sfruttato per poi svolgere delle predizioni: fornito un nuovo punto che rappresenta un dato non presente nel dataset con cui si è svolto il *training*, il modello deve essere in grado di collocarlo in uno dei cluster che ha individuato. Fra i task implementati, uno è dedicato alla creazione del modello e uno all'utilizzo di tale modello per svolgere le predizioni; è inoltre presente un task dedicato alla valutazione del modello, sfruttando il *Silhouette score* [18]. Nella versione classica dell'algoritmo la metrica utilizzata è la distanza euclidea, ma sono state proposte diverse alternative al fine di migliorare le performance [4, 3, 19]. La variante bisecting K-means [21] sfrutta l'algoritmo di base per implementare una procedura differente: inizialmente il dataset è considerato come un unico cluster e viene utilizzato K-means per dividerlo in due cluster, si ripete il processo un certo numero di volte e poi si tiene la sezione che presenta due

cluster con la più alta similarità fra gli elementi, infine vengono iterati i passaggi precedenti fino a raggiungere il numero di cluster desiderato. Come nel caso dell'algoritmo di base, anche bisecting K-means viene utilizzato per creare un modello con cui poi effettuare delle predizioni.

Decision Tree Classifier I classificatori ad albero sono modelli predittivi con una struttura assimilabile a un diagramma di flusso, ideali per il data mining supervisionato. Ogni nodo dell'albero è un test su un attributo del dato da classificare, ogni ramo è un possibile esito del test e ogni foglia indica l'etichetta di classificazione. Il nodo radice non ha rami in ingresso ed è infatti il punto di ingresso all'albero stesso. La classificazione viene effettuata eseguendo i test rappresentati dai nodi, di volta in volta scegliendo il test successivo sulla base del risultato del precedente, continuando fino all'ultimo livello (terminale), in cui tutte le tuple di dati in un nodo comprendono campioni di una sola classe. La costruzione del modello avviene seguendo la fase di generazione (*generation*) prima e di potatura (*pruning*) poi; nella prima fase tutte i dati di allenamento partono nel nodo radice, viene definito un criterio di divisione ed è selezionato il miglior attributo per effettuare la divisione, dopodiché i dati vengono partizionati nei nodi figli e la procedura è ripetuta fino a quando aggiungendo nodi foglia non si aumenta più la precisione dei nodi foglia. Nella seconda fase si punta a ridurre le dimensioni del modello in modo da renderlo più efficiente, in particolare vengono rimossi i sottoalberi che si sono creati per riflettere le caratteristiche di eventuale rumore nei dati o outliers [12]. Una volta che si ha un modello affidabile ed efficiente, lo si può utilizzare per la classificazione di dati.

Gaussian Mixture Model Si tratta di un modello statistico utilizzato per rappresentare la distribuzione di probabilità di un insieme di dati come una combinazione di più distribuzioni gaussiane. Un Gaussian Mixture Model (GMM) assume che i dati siano generati da un insieme di sottopopolazioni, ognuna che segue una distribuzione gaussiana. È definito da un numero specifico di componenti gaussiane, ognuna con la propria media e varianza; l'obiettivo dell'algoritmo è quello di costruire un modello sulla base di un certo insieme di dati, in modo tale da eseguire poi predizioni collocando nuovi dati nella gaussiana corretta (ovvero stimando la variabile latente) [13]. Il modello viene costruito utilizzando la classe offerta da Spark GaussianMixtureModel in un task, poi un secondo utilizza il modello per eseguire le predizioni.

K-Nearest Neighbors Classifier L'algoritmo k-nearest neighbors [6] è stato utilizzato per costruire un classificatore, quindi il task che lo implementa ha

l'obiettivo di restituire in output la classe a cui appartiene un oggetto che riceve in input. La classificazione avviene scegliendo la classe più comune fra i k oggetti con features più simili (e quindi "vicini") all'oggetto da classificare, dove k è parametro dell'algoritmo. Dal momento che il funzionamento dell'algoritmo si basa sul confronto della distanza fra l'oggetto da classificare e una serie di oggetti conosciuti, la fase di allenamento del modello consiste nell'estrazione delle caratteristiche dai dati dedicati al training, in modo da rappresentare ogni oggetto come un vettore in uno spazio n -dimensionale (dove n è il numero delle caratteristiche). Ogni vettore è in seguito assegnato a una classe sulla base della classificazione indicata nel dataset, si tratta dunque di un metodo di apprendimento supervisionato e non parametrico [20]: non vengono fatte assunzioni sulla distribuzione dei dati su cui opera l'algoritmo, rendendolo adatto alla classificazione di dati di cui non si conosce la distribuzione; occorre tenere presente, tuttavia, che il meccanismo con cui avviene la classificazione, comprende intrinsecamente il rischio che la classificazione sia svolta con bassa accuratezza nel caso in cui la distribuzione sia fortemente asimmetrica, perché gli esempi di una classe più frequente tendono a dominare la previsione di un nuovo esempio (tendono ad essere comuni tra i k vicini più prossimi a causa del loro grande numero), per questo sono state proposte delle alternative alla regola di base del voto a maggioranza dei k vicini [5]. Utilizzando le classi di Spark dedicate al modello KNNC — `KNNClassificationModel` e `KNNClassifier` — viene costruito il classificatore sulla base di un dataset in input, per poi utilizzare il modello per effettuare le predizioni.

Linear Discriminant Analysis Si tratta un algoritmo di riduzione della dimensionalità e di classificazione utilizzato nell'ambito dell'apprendimento supervisionato. L'obiettivo è trovare la combinazione lineare delle caratteristiche che massimizza la separazione tra le classi dei dati utilizzati nell'allenamento del modello, per farlo l'algoritmo LDA proietta i dati in uno spazio delle caratteristiche a dimensioni inferiori, senza perdere informazioni, in modo che le diverse classi siano il più separabili possibile lungo questa nuova dimensione. Vengono calcolate le medie delle caratteristiche per ciascuna classe, oltre alle matrici di dispersione intra-classe (*data spread* all'interno di una classe) e inter-classe (*data spread* fra le classi). Successivamente si calcolano le direzioni discriminanti, ovvero i vettori che massimizzano il rapporto tra la varianza inter-classe e la varianza intra-classe, i dati vengono proiettati su queste direzioni per ridurre la dimensionalità e separare le classi nel nuovo spazio delle caratteristiche. Dopo la proiezione dei dati, è possibile utilizzare un classificatore per classificare i nuovi dati in base alla loro posizione nello spazio delle caratteristiche ridotto. LDA è spesso utilizzato come tecnica di riduzione della

dimensionalità prima di applicare un classificatore, poiché aiuta a migliorare le prestazioni del modello riducendo il rischio di overfitting. È da notare che vengono fatte le assunzioni che i dati siano distribuiti normalmente e che le classi abbiano matrici simili [22]. Nel task implementato, vengono utilizzate le classi di clustering offerte da Spark LDA e LDAModel per allenare il modello ed effettuare le predizioni.

Principal Component Analysis In modo simile a LDA, questo algoritmo mira a ridurre la dimensionalità dei dati ed è quindi utilizzato nel *preprocessing*; l'idea di base è quella di estrarre le informazioni importanti dal dataset, per poi rappresentarle come variabili ortogonali (le componenti principali) ottenute come combinazioni lineari delle variabili originali, in modo tale da poter poi distinguere graficamente i diversi cluster. Per estrarre la prima componente si ricerca quella che ha varianza maggiore, perché in questo modo si estrae la maggior quantità di informazione dal dataset, per la seconda si pone il vincolo di ortogonalità con la prima e si cerca nuovamente quella con varianza maggiore (tolta la prima), reiterando il processo un numero pari alle componenti che si desidera estrarre [1]. Più nel dettaglio, il processo consiste in una prima normalizzazione dei dati (sottraendo la media e dividendo per la deviazione standard), per poi calcolare la matrice di covarianza (σ) dei dati, in seguito si calcolano autovalori e autovettori di σ , vengono selezionate le componenti principali estraendo gli autovettori con i corrispondenti autovalori più grandi, infine si proiettano i dati normalizzati sulle componenti principali estratte per ridurre la dimensionalità. Grazie alle classi di Spark PCA e PCAModel, il task esegue il *fitting* del modello e poi restituisce un dataset con le informazioni relative alle componenti principali.

ANalysis Of VAriance (ANOVA) ANOVA è una tecnica di inferenza statistica utilizzata per analizzare la differenza fra le medie di più gruppi differenti di dati, con il fine di descrivere le relazioni presenti fra le variabili presenti nel dataset. È particolarmente utile per analizzare i dati ottenuti da diverse misurazioni svolte in seguito a diversi generici "trattamenti" eseguiti nel corso di un esperimento, perché consente di valutare la significanza statistica dei "trattamenti" confrontando i dati con quelli di un ipotetico gruppo di controllo. Per utilizzare ANOVA si assume che ogni gruppo su cui si sono effettuate le misurazioni segua una distribuzione normale, le varianze delle popolazioni di provenienza dei gruppi siano uguali fra loro, le osservazioni sui vari gruppi siano indipendenti; l'ipotesi nulla è che tutte le medie delle diverse popolazioni considerate siano uguali, quindi che non ci sia una differenza significativa fra i diversi gruppi, mentre l'ipotesi alternativa è che almeno una media sia diffe-

rente, con un certo livello di significanza. La versione più semplice dell'analisi è *one-way ANOVA*, in cui viene esaminata l'influenza di una singola variabile indipendente, ed è quella eseguita dal task nel caso in cui il dataset in input sia composto da sole due colonne (una per la variabile dipendente e una per quella indipendente); la versione *two-way ANOVA*, che analizza l'influenza di due differenti variabili indipendenti, è eseguita se il dataset ha tre colonne, la versione *multi-way ANOVA* serve per l'analisi della varianza provocata da più di due variabili indipendenti simultaneamente ed è eseguita se il dataset ha almeno quattro colonne. L'algoritmo restituisce una tabella contenente i gradi di libertà, la somma dei quadrati, la media dei quadrati, l'*f-ratio*, il *p-value*, il numero di celle del dataset, la somma e la media dei loro valori. L'*f-ratio* è il rapporto fra la varianza tra i diversi gruppi, e la varianza all'interno dei singoli gruppi, mentre il *p-value* è la probabilità che l'*f-ratio* sia maggiore o uguale del valore atteso, calcolato prendendo direttamente il valore della *F-distribution* per il valore di significanza α , considerando i gradi di libertà di numeratore e denominatore dell'*f-ratio*; se *p-value* è minore di α l'ipotesi nulla è falsa, quindi c'è una differenza significativa dal punto di vista statistico fra le medie dei gruppi [16].

Linear Regression La regressione lineare è una tecnica statistica utilizzata per modellare la relazione tra una variabile dipendente e una o più variabili indipendenti, in modo da definire la variabile dipendente come una combinazione lineare di alcuni valori (le variabili indipendenti), detti predittori perché una volta individuata la relazione fra tali valori e la variabile dipendente, è possibile effettuare delle predizioni sulla base dei valori osservati. La forma più comune è la regressione lineare semplice (Simple Linear Regression (SLR)), che coinvolge una sola variabile indipendente, ma esistono diverse varianti della regressione lineare che possono essere utilizzate in diverse situazioni. L'obiettivo di SLR è quello di trovare una funzione lineare che predica il valore della variabile dipendente dato un certo valore di quella indipendente (ovvero data l'ascissa, il valore dell'ordinata deve essere una predizione accurata): le predizioni sono svolte sulla base di un solo predittore [17].

Un Generalized Linear Model (GLM) è una generalizzazione che mira a rendere più flessibile la regressione lineare classica, in cui le variabili coinvolte possono seguire una distribuzione arbitraria, e viene utilizzata una *link function* che mette in relazione i predittori con la media della distribuzione della variabile dipendente; anziché far variare linearmente la variabile dipendente stessa, in questo modello è una funzione di tale variabile che varia linearmente con i predittori. La regressione lineare è un GLM in cui la distribuzione della variabile dipendente è una normale con varianza costante e la *link function* è

l'identità [7]. Il task implementato offre la possibilità di scegliere, con un opportuno parametro, se utilizzare la SLR o la Generalized Linear Regression (GLR) che utilizza un GLM.

Logistic Regression Nel caso in cui si abbia a che fare con delle variabili che possono assumere soltanto due valori (circostanza frequente nei classificatori), è possibile utilizzare un particolare tipo di regressione — la regressione logistica — che utilizza la funzione logit per associare ad un vettore in ingresso un valore di probabilità di appartenenza ad una delle due classi possibili [7, 9]. Per costruire un modello ML, durante la fase di addestramento (supervisionato) si utilizzano i dati dedicati per trovare i coefficienti della funzione logit che massimizzano la corrispondenza fra i dati e la loro classe di appartenenza, poi viene stabilito il *decision boundary* per separare le diverse classi nello spazio delle features. Una volta allenato il modello, si può utilizzare per effettuare classificazione presentando in ingresso nuovi dati: l'output del modello è la probabilità che il vettore in ingresso rappresenti un oggetto appartenente a una delle due classi. I task implementati offrono anche la possibilità di vedere i parametri necessari alla valutazione del modello (precision, recall, f-score) e l'evoluzione della precisione del modello durante la fase di addestramento.

Capitolo 3

Recupero e implementazione degli algoritmi

La prima fase del lavoro è consistita nel recupero di alcuni algoritmi sviluppati con tecnologia Spark sfruttando la libreria MLlib, originariamente implementati nell'ambito del progetto EVOTION [2]. Affinché i singoli Spark task fossero eseguibili singolarmente, dopo una prima analisi del codice già sviluppato, si è resa necessaria l'estrazione delle singole classi Java che implementano i diversi algoritmi e la contestuale creazione di un progetto autonomo per ogni algoritmo. L'autonomia dei singoli progetti è un requisito necessario per avere la possibilità di creare in maniera semplice i cosiddetti *fat jar* (o *uber jar*) — maggiori dettagli in seguito — e dal momento che le classi sono state estratte da un progetto più grande, è stato necessario risolvere le dipendenze perché ogni algoritmo potesse importare tutte le classi e librerie esterne di cui necessitava. Un'ulteriore attività svolta è stata quella di aggiornamento della versione di Spark dalla 2.2.0 alla 3.5.0, mantenendo Java 8; dal momento che numerosi algoritmi fra quelli coinvolti nel lavoro utilizzano la classe "VectorDisassembler", realizzata in Scala 2.11 per Spark 2.2.0 da un singolo sviluppatore e non più aggiornata, è stata creata una *fork* della classe in modo da poter aggiornare la versione di Scala e così rendere il codice compatibile con Spark 3.5.0.

Maven I progetti Java sono stati gestiti con Apache Maven, uno strumento per l'amministrazione di progetti software, quindi per risolvere le dipendenze e aggiornare la versione di Spark, l'attività svolta è stata quella di scrivere un nuovo file Project Object Model (POM) per ogni algoritmo; tale file è in formato eXtensible Markup Language (XML) e comprende sia i dettagli di configurazione del progetto, sia le informazioni necessarie a Maven per realizzare il pacchetto contenente l'applicazione Java. Il file POM contiene delle sezioni che sono dedicate alla struttura del progetto, ovvero in cui sono indicate le relazioni fra le diverse

classi; una sezione dedicata alle proprietà del progetto (strettamente dipendenti dal linguaggio e dalle tecnologie utilizzate); una sezione dedicata alla fase di costruzione del pacchetto dell'applicazione Java, dove possono essere configurati vari plugin; un'ultima sezione in cui sono indicate tutte le dipendenze del progetto. All'interno di un singolo progetto possono esserci diversi file POM, ognuno che gestisce una parte del codice, a diversi livelli di dettaglio: un file più generale può indicare soltanto l'identificativo del gruppo (package) di cui fa parte il progetto, l'identificativo dell'artefatto e i diversi moduli che lo compongono; i file dei singoli moduli poi possono specificare i plugin necessari alla costruzione del file *jar*, così come le dipendenze. Alcuni algoritmi prevedono la presenza di un task Spark dedicato soltanto all'allenamento del modello e un task diverso per effettuare predizioni utilizzando il modello allenato; in questo caso è stato organizzato un unico progetto in cui i due task sono moduli di uno stesso artefatto generale. Un esempio di file POM generale è riportato nell'appendice A, mentre un esempio di un file di un singolo modulo, contenente sia i plugin sia le dipendenze, è riportato nell'appendice B.

Uber jar Gli applicativi Java sono solitamente "confezionati" in file archivio, chiamati JAR (da Java Archive), che comprendono sia i file di diverse classi Java, sia i relativi metadati e tutte le risorse che utilizzano; queste generiche risorse possono essere sia file di testo o multimediali, sia altri file JAR. I file JAR sono un aspetto fondamentale del linguaggio Java, sia per quanto riguarda la facilità di distribuzione sia per la sicurezza, infatti è possibile inserire in un unico file, detto *uber jar* o *fat jar*, tutti i file delle classi sviluppate per l'applicativo, uniti a tutte le classi che costituiscono le dipendenze, rendendo l'archivio un file completamente autonomo, ovvero eseguibile senza necessità di importare nessuna classe esterna.

Aggiornamento In particolare, l'attività di aggiornamento e riorganizzazione del codice si è svolta in più fasi in cui sono state prima risolte le dipendenze delle classi relative ai singoli algoritmi, svolgendo un test con un dataset di esempio per ciascun task, lanciando l'esecuzione direttamente dall'interno dell'ambiente di sviluppo, in seguito è stata aggiornata la versione di Spark, e sono state svolte minime modifiche al codice Java delle classi per risolvere alcuni problemi di compatibilità relativi all'aggiornamento, solitamente dovuti al fatto che nella nuova versione di Spark alcune classi sono state spostate in librerie differenti rispetto alla versione 2.2.0. Per l'aggiornamento della classe "VectorDisassembler", dopo aver effettuato la *fork* del codice, si è aggiornato il file POM, poi si è costruito il nuovo file *jar* che è stato in seguito importato da

tutti i task che lo richiedevano. Una volta aggiornati i file di configurazione di tutti i progetti relativi ai singoli algoritmi, sono stati costruiti i file *uber jar*.

Capitolo 4

Conclusioni

Appendice A

POM generale

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-
  instance"
    xsi:schemaLocation="http://maven.apache.org/
      POM/4.0.0 https://maven.apache.org/xsd/
      maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>it.unimi.evotion.tasks</groupId>
  <artifactId>kmeans-task</artifactId>
  <version>1.0</version>
  <packaging>pom</packaging>

  <modules>
    <module>kmeans</module>
    <module>kmeans_model</module>
    <module>kmeans_predict</module>
  </modules>
</project>
```

Appendice B

POM di un singolo modulo

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-
    instance"
  xsi:schemaLocation="http://maven.apache.org/
    POM/4.0.0 http://maven.apache.org/xsd/
    maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>it.unimi.evotion.tasks</groupId>
    <artifactId>kmeans-task</artifactId>
    <version>1.0</version>
  </parent>

  <artifactId>kmeans</artifactId>
  <name>KMeans</name>
  <version>1.0</version>

  <properties>
    <maven.compiler.source>1.8</maven.compiler.
      source>
    <maven.compiler.target>1.8</maven.compiler.
      target>
    <project.build.sourceEncoding>UTF-8</project.
      build.sourceEncoding>
```

```

        <spark.version>3.5.0</spark.version>
        <scala.version>2.12</scala.version>
    </properties>

    <build>
        <plugins>
            <plugin>
                <artifactId>maven-assembly-plugin</
                artifactId>
                <configuration>
                    <archive>
                        <manifest>
                            <addClasspath>true</
                            addClasspath>
                            <mainClass>it.unimi.
                                evotion.tasks.kmeans.
                                Kmeans</mainClass>
                        </manifest>
                    </archive>
                    <descriptorRefs>
                        <descriptorRef>jar-with-
                            dependencies</descriptorRef
                            >
                    </descriptorRefs>
                </configuration>
            <executions>
                <execution>
                    <id>make-assembly</id>
                    <phase>package</phase>
                    <goals>
                        <goal>single</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
        <plugin>
            <groupId>org.apache.logging.log4j</
            groupId>
            <artifactId>log4j-transform-maven-
                plugin</artifactId>
            <version>0.1.0</version>

```



```

        <executions>
            <execution>
                <goals>
                    <goal>process-classes </
                        goal>
                </goals>
            </execution>
        </executions>
    </plugin>
</plugins>
</build>

<dependencies>
    <dependency>
        <groupId>org.apache.spark </groupId>
        <artifactId>spark-core_${scala.version} </
            artifactId>
        <version>${spark.version} </version>
        <scope>provided </scope>
    </dependency>
    <dependency>
        <groupId>org.apache.spark </groupId>
        <artifactId>spark-sql_${scala.version} </
            artifactId>
        <version>${spark.version} </version>
        <scope>provided </scope>
    </dependency>
    <dependency>
        <groupId>org.apache.spark </groupId>
        <artifactId>spark-mllib_${scala.version} </
            artifactId>
        <version>${spark.version} </version>
        <scope>provided </scope>
    </dependency>
    <dependency>
        <groupId>org.apache.spark </groupId>
        <artifactId>spark-streaming_${scala.
            version} </artifactId>
        <version>${spark.version} </version>
        <scope>provided </scope>
    </dependency>

```

```

<dependency>
  <groupId>org.apache.logging.log4j </groupId>
  <
    <artifactId>log4j-api </artifactId>
    <version>2.22.1 </version>
  </dependency>
<dependency>
  <groupId>org.apache.logging.log4j </groupId>
  <
    <artifactId>log4j-core </artifactId>
    <version>2.22.1 </version>
  </dependency>
<dependency>
  <groupId>org.apache.spark.ml.feature </
    groupId>
  <artifactId>VectorDisassembler </artifactId>
  <
    <version>1.0 </version>
  </dependency>
</dependencies>
</project>

```


Glossario

Amazon S3 File system distribuito offerto da Amazon. III

Apache Airflow Scheduler sviluppato da Airbnb e reso *open source*. III

Big Data Insiemi di dati ad alta dimensionalità, spesso non strutturati, prodotti in maniera continuativa da un servizio o da dispositivi IoT. III, 1

Bigtable Sistema di archiviazione distribuito per dati strutturati. III

cluster Insieme di risorse computazionali. III, 20, 21

Container Unità di virtualizzazione software che contiene un software unitamente alle sue dipendenze, senza virtualizzare un intero sistema operativo. 20

data pipeline Sequenza di task che effettuano operazioni sui dati, in cui l'output di ogni task è usato come input dal successivo. 21

Docker Strumento per eseguire e amministrare i Container. III

file system Struttura dati utilizzata per la gestione dei dati archiviati su un supporto elettronico. III, 20

Hadoop Yarn Yet Another Resource Negotiator, è la tecnologia di gestione delle risorse e pianificazione dell'esecuzione task nel framework di elaborazione distribuita open source Hadoop. III, 2

Kubernetes Orchestratore per gestire un cluster composto da Container. III, IV, 2

Luigi Scheduler sviluppato da Spotify e reso *open source*. III

MapReduce Modello di programmazione per processare in maniera semplice operazioni, svolte su grandi dataset, eseguite su cluster di grandi dimensioni. III

scheduler Strumento per la gestione di flussi di lavoro, nell'ambito specifico di questo elaborato, si intende per l'ingegnerizzazione di data pipeline. III, 20

Acronimi

ANOVA ANalysis Of VAriance. 7, 8

API Application Programming Interface. IV

DL Deep Learning. IV, 1

DoS Denial of Service. 2

EMR Elastic MapReduce. III

GLM Generalized Linear Model. 8, 9

GLR Generalized Linear Regression. 9

GMM Gaussian Mixture Model. 5

IoT Internet of Things. 1

ML Machine Learning. III, IV, 1, 2, 4, 9

MLlib Machine Learning library. III, 1, 10

POM Project Object Model. 10, 11

SLR Simple Linear Regression. 8, 9

XML eXtensible Markup Language. 10

Bibliografia

- [1] Hervé Abdi e Lynne J. Williams. “Principal Component Analysis”. In: *WIREs Computational Statistics* 2.4 (2010), pp. 433–459. issn: 1939-5108, 1939-0068. doi: 10.1002/wics.101.
- [2] Marco Anisetti et al. *Big Data Analytics Engine , Deliverable D5.5 to the EVOTION-727521 Project Funded by the European Union*. UNIMI, Italy, 2018.
- [3] Li Chen et al. “Fast Kernel k -Means Clustering Using Incomplete Cholesky Factorization”. In: *Applied Mathematics and Computation* 402 (2021), p. 126037. issn: 00963003. doi: 10.1016/j.amc.2021.126037.
- [4] Xiaohui Chen e Yun Yang. “Diffusion K-means Clustering on Manifolds: Provable Exact Recovery via Semidefinite Relaxations”. In: *Applied and Computational Harmonic Analysis* 52 (2021), pp. 303–347. issn: 10635203. doi: 10.1016/j.acha.2020.03.002.
- [5] D. Coomans e D.L. Massart. “Alternative K-Nearest Neighbour Rules in Supervised Pattern Recognition”. In: *Analytica Chimica Acta* 136 (1982), pp. 15–27. issn: 00032670. doi: 10.1016/S0003-2670(01)95359-0.
- [6] T. Cover e P. Hart. “Nearest Neighbor Pattern Classification”. In: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27. issn: 0018-9448, 1557-9654. doi: 10.1109/TIT.1967.1053964.
- [7] Annette J. Dobson e Adrian G. Barnett. *An Introduction to Generalized Linear Models*. Fourth edition. Chapman & Hall/CRC Texts in Statistical Science Series. Boca Raton: CRC Press, Taylor & Francis Group, 2018. isbn: 978-1-138-74151-5 978-1-138-74168-3.
- [8] Jianqing Fan, Fang Han e Han Liu. “Challenges of Big Data Analysis”. In: *National Science Review* 1.2 (2014), pp. 293–314. issn: 2053-714X, 2095-5138. doi: 10.1093/nsr/nwt032.
- [9] David W. Hosmer, Stanley Lemeshow e Rodney X. Sturdivant. *Applied Logistic Regression*. Third edition. Wiley Series in Probability and Statistics 398. Hoboken, New Jersey: Wiley, 2013. isbn: 978-1-118-54835-6.

- [10] Abiodun M. Ikotun et al. “K-Means Clustering Algorithms: A Comprehensive Review, Variants Analysis, and Advances in the Era of Big Data”. In: *Information Sciences* 622 (2023), pp. 178–210. issn: 00200255. doi: 10.1016/j.ins.2022.11.139.
- [11] Jasmin Praful Bharadiya. “Machine Learning and AI in Business Intelligence: Trends and Opportunities”. In: *International Journal of Computer (IJC)* 48.1 (2023), pp. 123–134.
- [12] N.A. Priyanka e Dharmender Kumar. “Decision Tree Classifier: A Detailed Survey”. In: *International Journal of Information and Decision Sciences* 12.3 (2020), p. 246. issn: 1756-7017, 1756-7025. doi: 10.1504/IJIDS.2020.108141.
- [13] Douglas Reynolds. “Gaussian Mixture Models”. In: *Encyclopedia of Biometrics*. A cura di Stan Z. Li e Anil K. Jain. Boston, MA: Springer US, 2015, pp. 827–832. isbn: 978-1-4899-7487-7 978-1-4899-7488-4. doi: 10.1007/978-1-4899-7488-4_196.
- [14] Salman Salloum et al. “Big Data Analytics on Apache Spark”. In: *International Journal of Data Science and Analytics* 1.3-4 (2016), pp. 145–164. issn: 2364-415X, 2364-4168. doi: 10.1007/s41060-016-0027-9.
- [15] Iqbal H. Sarker. “Machine Learning for Intelligent Data Analysis and Automation in Cybersecurity: Current and Future Prospects”. In: *Annals of Data Science* 10.6 (2023), pp. 1473–1498. issn: 2198-5804, 2198-5812. doi: 10.1007/s40745-022-00444-2.
- [16] Henry Scheffé. *The Analysis of Variance*. Wiley classics library ed. A Wiley Publication in Mathematical Statistics. New York: Wiley-Interscience Publication, 1999. isbn: 978-0-471-34505-3 978-0-471-75834-1.
- [17] George A. F. Seber e Alan J. Lee. *Linear Regression Analysis*. 2nd ed. Hoboken: John Wiley & Sons, 2012. isbn: 978-1-118-27442-2.
- [18] Ketan Rajshekhar Shahapure e Charles Nicholas. “Cluster Quality Analysis Using Silhouette Score”. In: *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*. sydney, Australia: IEEE, 2020, pp. 747–748. isbn: 978-1-72818-206-3. doi: 10.1109/DSAA49011.2020.00096.
- [19] Xiaobo Shen et al. “Compressed K-Means for Large-Scale Clustering”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 31.1 (2017). issn: 2374-3468, 2159-5399. doi: 10.1609/aaai.v31i1.10852.
- [20] Sidney Siegel. “Nonparametric Statistics”. In: *The American Statistician* 11.3 (1957), pp. 13–19. issn: 0003-1305, 1537-2731. doi: 10.1080/00031305.1957.10501091.

- [21] Michael Steinbach, George Karypis e Vipin Kumar. “A Comparison of Document Clustering Techniques”. In: (2000).
- [22] Alaa Tharwat et al. “Linear Discriminant Analysis: A Detailed Tutorial”. In: *AI Communications* 30.2 (2017), pp. 169–190. issn: 18758452, 09217126. doi: 10.3233/AIC-170729.
- [23] Xue-Wen Chen e Xiaotong Lin. “Big Data Deep Learning: Challenges and Perspectives”. In: *IEEE Access* 2 (2014), pp. 514–525. issn: 2169-3536. doi: 10.1109/ACCESS.2014.2325029.